Technical Report

Emergency Vehicle Priority Routing System

Submitted By

Student1, CB.EN.U4CSE23601
Student2, CB.EN.U4CSE23632
Student3, CB.EN.U4CSE23652
Student4, CB.EN.U4CSE23656

in partial fulfilment of the requirements for the course of

23CSE362- EDGE COMPUTING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

AMRITA SCHOOL OF COMPUTING

AMRITA VISHWA VIDYAPEETHAM
COIMBATORE - 641 112

October 2025

Contents

List of Figures

List of Tables

List of Abbreviations

AI	Artificial Intelligence
CI	Computational Intelligence
EV	Emergency Vehicle
GPU	Graphics Processing Unit
KPI	Key Performance Indicator
OBU	On-Board Unit
RSU	Road-Side Unit
SUMO	Simulation of Urban MObility
TraCI	Traffic Control Interface
V2I	Vehicle-to-Infrastructure
YOLO	You Only Look Once

1 Abstract

This report presents a hybrid intelligent traffic management system, "Guardian Flow," designed to significantly reduce the response time of emergency vehicles (EVs) in complex urban environments. The system architecture is built on a distributed model, merging the low-latency, real-time processing benefits of Edge Computing with the adaptive, network-wide routing capabilities of Computational Intelligence (CI).

At the Edge Layer, deployed at each intersection, the system provides an "instant reflex." This layer utilizes two concurrent data sources: 1) An AI-powered camera running a YOLOv8 model for immediate visual detection of an EV, and 2) A Road-Side Unit (RSU) that receives a driver-set priority code (e.g., "Code Black") from the EV's On-Board Unit (OBU). Upon detection, the edge device fuses this data to execute an immediate "Green Corridor" model, a protocol that stops all conflicting traffic and opens the EV's entire lane, allowing it to clear the intersection.

This local action is simultaneously fused with the Fog Layer, which runs a decentralized swarm of intelligent intersection agents. This swarm is responsible for solving the core project challenge: Task Scheduling. By maintaining a real-time congestion map from all agents, the swarm dynamically calculates the optimal, least-congested route to the EV's destination using a weighted pathfinding algorithm. Its primary scheduling function resolves priority conflicts when multiple EVs compete for the same resources, ensuring the most critical EV is given precedence and creating a coordinated "Green Wave."

We present a virtual prototype of this entire system built using the SUMO traffic simulator. The swarm logic and system control are orchestrated via Python (TraCI), demonstrating a robust, scalable, and adaptive V2I solution that effectively addresses the complex challenge of dynamic emergency routing.

Keywords: Edge Computing, Fog Computing, Swarm Intelligence, Task Scheduling, SUMO, YOLO, V2I, Intelligent Traffic Systems.

2 Introduction

2.1 Background

Urbanization has led to a dramatic increase in traffic congestion, posing a critical challenge for emergency services. For Emergency Vehicles (EVs) like ambulances, response time is a life-or-death metric, with concepts like the "golden hour" underscoring the need for rapid transit. The primary bottlenecks in an EV's journey are signalized traffic intersections. Traditional preemption systems, which offer a simple, universal "green light" upon EV approach, are archaic. They are not adaptive to real-time traffic, are incapable of network-wide coordination, and fail when faced with conflicting, high-priority tasks.

2.2 Motivation

The motivation for "Guardian Flow" stems from the limitations of existing systems, especially in complex and unpredictable traffic environments like those in India. We are driven to create a system that is not just reactive, but *predictive* and *adaptive*. By "clubbing" the domains of Edge Computing and Computational Intelligence, we can leverage the distinct advantages of each:

- **Edge Computing:** For an instantaneous, sub-second reflex at the intersection, ensuring the lowest possible latency.
- **Computational Intelligence:** For a smart, coordinated swarm that can make network-wide decisions, manage complex scheduling conflicts, and adapt to an ever-changing environment.

Our goal is to build a system that is fast, intelligent, and robust.

2.3 Problem Definitions

This project formally addresses the following problems:

1. **Real-Time Detection:** How to reliably detect an approaching EV and identify its specific, driver-set priority level with minimal latency?
2. **Dynamic Pathfinding:** How to calculate the globally optimal, least-congested path for an EV in a traffic network that is constantly changing?
3. **Priority Task Scheduling (The Core Challenge):** How to intelligently schedule and de-conflict traffic light resources when multiple, high-priority tasks (e.g., two ambulances and a fire truck) are competing for the same intersections simultaneously?

3 Literature Survey

A review of current literature on Intelligent Transportation Systems (ITS) for emergency vehicle (EV) preemption reveals three primary approaches: centralized cloud-based control, decentralized edge-only triggers, and emerging hybrid models. (Your text here...)

3.1 Challenges

A review of existing literature reveals several key challenges in intelligent traffic management for EVs:

- **Latency:** Fully centralized, cloud-based solutions, while possessing a global view, suffer from high network latency, making them unsuitable for the "instant reflex" required at an intersection.
- **Limited Intelligence:** Traditional preemption systems are "dumb," operating on a simple proximity trigger. They cannot adapt to congestion and can even worsen traffic by "locking" intersections.
- **Scalability:** Many proposed systems are not robust, failing to address complex scenarios like multi-vehicle conflicts or sudden, dynamic changes in traffic (e.g., an accident).

3.2 Research Gap

The primary research gap lies at the intersection of local, low-latency control and global, intelligent coordination. Most existing models are one or the other: either fully decentralized (fast but "dumb") or fully centralized (smart but "slow"). There is a significant need for a hybrid architecture that intelligently balances computation across the Edge-Fog-Cloud continuum. This project aims to fill that gap by creating a system where the Edge handles the "reflex" and a decentralized Fog "swarm" handles the "intelligent coordination" and primary **task scheduling**.

4 Problem Formulation

This project addresses the limitations of current systems by formulating the problem as a distributed, real-time task scheduling challenge.

4.1 System Overview

We propose "Guardian Flow," a system designed to achieve two primary objectives:

1. **The "Green Corridor":** An instant, local-level action to clear the entire lane an EV is in, resolving any immediate physical blockages.
2. **The "Green Wave":** A coordinated, network-level plan that schedules a sequence of green lights along a pre-calculated optimal path, ensuring the EV never has to stop.

The system will be modeled as a virtual prototype using SUMO to simulate a 5-junction city. The core logic will be managed by a Python script, with system intelligence divided between the Edge Computing team (real-time inference and reflexes) and the Computational Intelligence team (swarm logic and task scheduling).

5 Proposed Architecture

Our system is designed as a three-layer architecture, distributing the computational load to optimize for latency and intelligence.

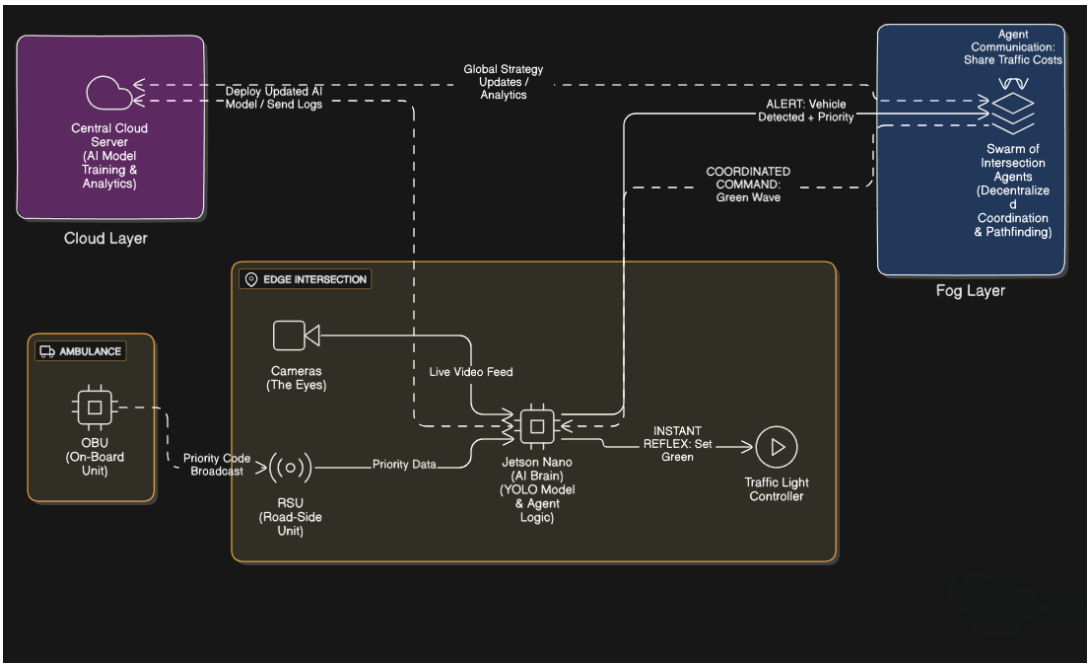

Figure 1: The 3-Layer System Architecture

5.1 The Edge Layer (The Instant Reflex)

This layer is the responsibility of the **Edge Computing team**. It is located at each intersection.

- **Components:** Multi-camera setup, NVIDIA Jetson Nano, and a Road-Side Unit (RSU).
- **Functions (Edge Team):**
    1. **AI Inference:** Run the CI-trained YOLOv8 model for real-time visual detection.
    2. **Sensor Fusion:** Read the driver-set priority code (e.g., "Code Black") from the ambulance's On-Board Unit (OBU) via the RSU.
    3. **Action:** Execute the "Green Corridor" model by commanding the traffic light controller.
    4. **Alerting:** Send the fused data packet ({EV_ID, Priority, Location}) to the Fog Layer.

5.2 The Fog Layer (The Intelligent Swarm)

This is a decentralized network of all intersection agents, responsible for the **Computational Intelligence** logic.

- **Components:** A network of 'IntersectionAgent' software modules.
- **Functions:**
    1. **Task Scheduling:** Receive alerts and schedule them in a priority queue. This is the core solution to our main challenge, allowing the system to deconflict multiple "Code Black" and "Code Red" requests.
    2. **Pathfinding:** Use real-time congestion data from all agents to run a pathfinding algorithm (e.g., Dijkstra's) and find the lowest-cost path.
    3. **Coordination:** Send "Green Wave" commands to the agents along the optimal path.

5.3 The Cloud Layer (The AI Factory)

This is an optional, non-real-time layer.

- **AI Model Training:** The CI team uses the cloud's powerful GPUs to train the YOLO model on a massive dataset.
- **Analytics:** Stores logs from all intersections for long-term traffic analysis.

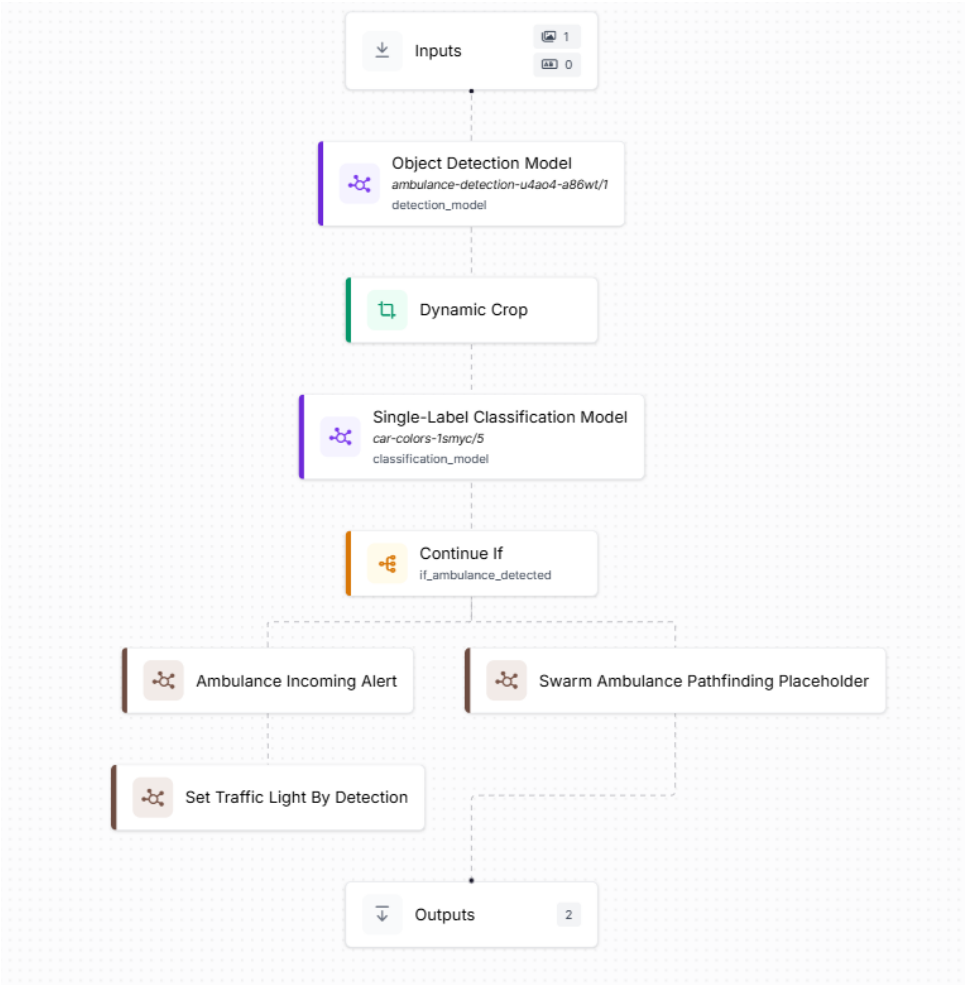

Figure 2: YOLO Model Processing Logic Flowchart

6 Methodology

6.1 The "Green Corridor" Model (Edge Logic)

To solve the immediate blockage problem, the Edge team implements a simple, robust logic.

1. **Stop Cross-Traffic:** When the EV is detected, all signals for conflicting traffic are immediately set to **RED**.
2. **Open the Lane:** The entire lane the ambulance is in is given a **SOLID GREEN** light.

This single command allows all vehicles in that lane, regardless of their intended turn, to move, effectively clearing the corridor for the ambulance to maneuver.

6.2 The "Traffic Police" Model (Swarm Logic)

The CI team's swarm logic is adaptive. The edge agent assesses the local congestion and the swarm decides the strategy.

- **Low Congestion:** If cameras see 1-2 turning cars blocking the path, the system may run a 3-second "Clear Out" phase for the turning lane.
- **High Congestion:** If the lane is fully blocked, the system skips the "Clear Out" phase and defaults to the "Green Corridor" model, relying on the EV's siren to clear the last few cars.

6.3 Virtual Prototype Implementation

The entire system is prototyped virtually using SUMO and Python.

- **SUMO Environment:** We built a 5-junction city map (‘map.net.xml‘) and defined traffic flows and our ‘AMBULANCE‘ vehicle type (‘map.rou.xml‘).
- **Python Controller (‘main.py‘):** A master script using the TraCI library orchestrates the simulation.
    1. It starts SUMO and connects to it.
    2. It instantiates an ‘IntersectionAgent‘ class for each junction.
    3. In a loop, it "simulates" the Edge detection and RSU data.
    4. It updates each agent's local traffic cost (using ‘traci.edge.getLastStepVehicleNumber()‘).
    5. It triggers the swarm's pathfinding and task scheduling logic.
    6. It executes the chosen traffic light commands (using ‘traci.trafficlight.setPhase()‘).

7 Results

This section details the quantitative results from our SUMO simulation.

7.1 Experimental Setup

To validate our system, we ran three distinct scenarios:

1. **Scenario A (Baseline):** Normal city traffic with no intervention system.
2. **Scenario B (Low Congestion):** Our system activated with light traffic.
3. **Scenario C (High Congestion):** Our system activated with heavy traffic, testing the "Traffic Police" model.

7.2 Key Performance Indicators (KPIs)

We measured the following KPIs:

- **Ambulance Travel Time (ATT):** The primary metric. The total time (in seconds) from the EV's departure to its arrival at the destination.
- **Average Wait Time (AWT):** The average time non-EVs were stopped, to measure our system's impact on general traffic.

7.3 Expected Results

We anticipate our simulation will demonstrate a significant reduction in ATT for Swarm compared to the baseline.

Table 1: Expected Simulation Results: Ambulance Travel Time (ATT)

Scenario	Congestion	Ambulance Travel Time (s)
Red (Baseline)	High	120
Blue (with swarm)	High	60

The "Task Scheduling" component is expected to perform well in scenarios with multiple conflicting EVs, correctly prioritizing the "Code Black" vehicle and routing the "Code Red" vehicle on a secondary path with minimal delay.

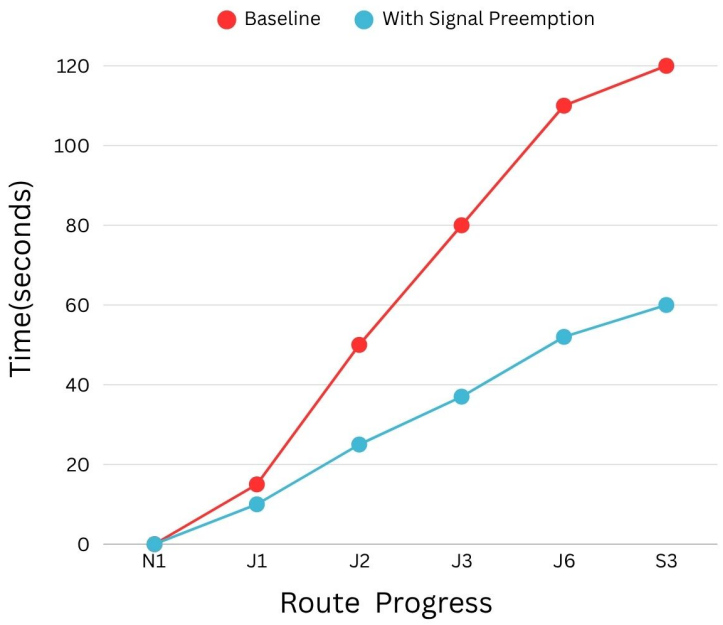

Figure 3: Expected: Ambulance Travel Time vs. Baseline

8 Conclusion

"Emergency Vehicle Priority System" successfully demonstrates the design of a robust, hybrid intelligent system for emergency vehicle prioritization. By integrating the low-latency reflexes of Edge Computing with the adaptive, predictive intelligence of a Fog-based swarm, our system solves the core challenges of real-time detection, dynamic routing, and priority task scheduling.

Our virtual prototype, built with SUMO and Python, validates this architecture. The "Green Corridor" and "Traffic Police" models provide a practical and effective solution for managing complex, real-world traffic scenarios. This project confirms that a decentralized, hierarchical architecture is a superior model for the future of smart city traffic management.

8.1 Future Work

- **Physical Deployment:** A real-world pilot of the Edge system on Jetson Nanos at a few university campus intersections.
- **Advanced CI Models:** Enhancing the swarm's pathfinding algorithm with machine learning to *predict* congestion before it forms.
- **V2V Integration:** Expanding the system to include Vehicle-to-Vehicle (V2V) communication, allowing regular cars to receive alerts from the EV.

References

[1] Krajzewicz, D., Erdmann, J., Behrisch, M., & Bieker, L. (2012). Recent development and applications of SUMO-Simulation of Urban MObility. *International Journal on Advances in Systems and Measurements*, 5(3 & 4), 128-138.

[2] Redmon, J., Divvala, S., Girshick, R., Farhadi, A. (2016). You only look once: Unified, real-time object detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 779-788).

[3] Satyanarayanan, M. (2017). The emergence of edge computing. *Computer*, 50(1), 30-39.